5树上操作

树的重心

U164672 树的重心

题目描述：

给定一颗树，树中包含 n 个结点（编号$1 \sim n$）和$n-1$条无向边。

请你找到树的重心，并输出将重心删除后，剩余各个连通块中点数的最大值。

重心定义：重心是指树中的一个结点，如果将这个点删除后，剩余各个连通块中点数的最大值最小，那么这个节点被称为树的重心。

输入格式

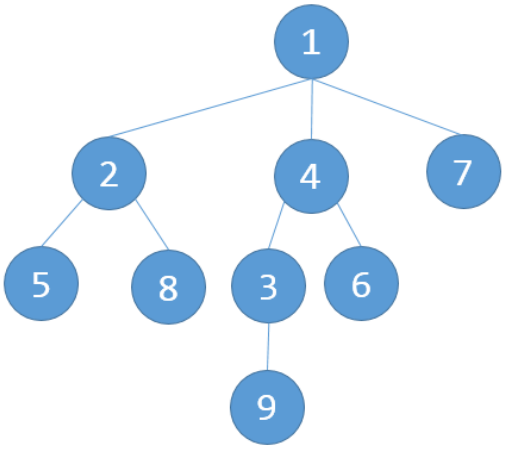
第一行包含整数n，表示树的结点数。

接下来$n-1$行，每行包含两个整数a和 b，表示点a和点 b之间存在一条边。

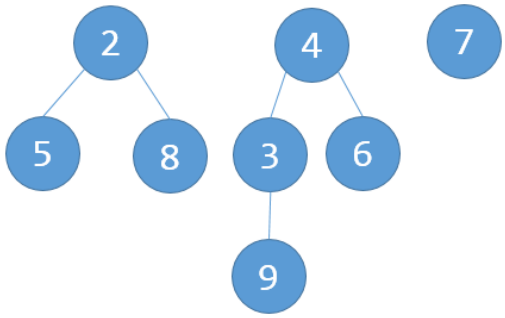
输出格式

第一行：一个整数 p， 表示重心的最小结点

第二行：一个整数 m，表示将重心删除后，剩余各个连通块中点数的最大值。



原图



删除结点1

从图上可以分析出：

1. 如果删除结点 1 ，子树的最大值为 4
2. 如果删除结点 2 ， 子树的最大值为6
3. 如果删除结点 3 ，子树的最大值为7
4. … …

综上所述，删除结点 1 ，子树的最大值最小为4，结点1为树的重心。

参考代码：

```cpp
#include<bits/stdc++.h>
using namespace std;
const int N=100010, M=2*N;
int h[N],e[M],ne[M],idx;
int ans[N],son[N];
int n;
void add(int x,int y) {
    e[++idx]=y; ne[idx]=h[x]; h[x]=idx;
}
int dfs(int x,int fa) {
    int sum=1;      //初始化当前连通块为1，即父结点自己
    for(int i=h[x];i;i=ne[i]) {
        int y=e[i];
        if(y==fa)   continue;       //不往回走
        int s=dfs(y,x);
        sum+=s;                     //累加子树数量
        son[x]=max(son[x],s);       //计算最大子树数量
    }
    ans[x]=max(son[x],n-sum);       //删掉该点后能得到最大的连通块数量
    return sum;                     //返回子树数量
}

int main() {
    cin>>n;
    for(int i=1;i<n;i++) {
        int x,y;    scanf("%d%d",&x,&y);
        add(x,y);   add(y,x);
    }
    dfs(1,0);


    int s=1e9, pos=0;
    for(int i=1;i<=n;i++) {
        if(s>ans[i]) {
            s=ans[i];   //计算最大值的最小值
            pos=i;      //计算最小的重心结点
        }
    }
    cout<<pos<<endl;
    cout<<s;
    return 0;
}
```

性质：

1. 树中所有点到某个点的距离和中，到重心的距离和是最小的；如果有两个重心，那么他们的距离和一样。
2. 把两个树通过一条边相连得到一个新的树，那么新的树的重心在连接原来两个树的重心的路径上。
3. 在一棵树添加或删除一个叶子，那么它的重心最多只移动一条边的距离。

树的直径

定义：在一棵树中，每一条边都有权值，树中的两个点之间的距离，定义为连接两点的路径上边权之和，那么他们之间的距离，就被称之为**树的直径**。直径也称为**树的最长链**。需要注意的是：直径既是一个数值的概念，也可代指一条路径。

树的直径一般有两种求法，时间复杂度都是$O(N)$。我们假设树以N个点$N-1$条边的无向图的形式给出，并存储在邻接表中。

树形DP 求树的直径

树形dp就是在树结构上所做的动态规划，通常树形dp是基于dfs来实现的。在用dfs遍历节点并维护信息的时候采用"后序"的思想：对于每个节点，都要用其所有儿子节点的信息来更新它自身的信息，只有当所有儿子节点的信息都得到确认之后，父节点的信息才能得到确认。

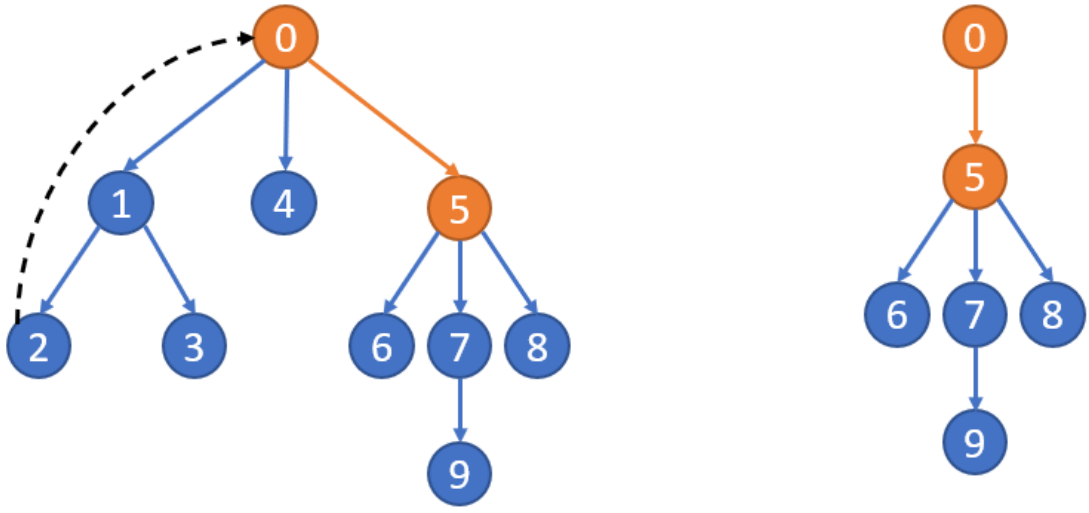
设$d[x]$表示从结点 x 出发走向以 x 为根的子树，能够到达的最远结点的距离。

设点(x,y)的权值为 $w(x,y)$。

设$f[x]$表示 以x为根结点的一条最长路径的距离。

DP关系式：$f[x]=max(d[x]+w(x,y)+d[y])$。

整棵树的最大距离为：$ans=max(f_i,f_2,\dots,f_n)$



经过点 0 时，直径的两种可能性

代码模板

```
void dp(int x) {
    vis[x]=1;
    for(int i=h[x];i;i=ne[i]) {
        int y=e[i];
        if(vis[y])  continue;
        dp(y);  //后序遍历的方式
        f[x] = d[x]+d[y]+w[i];
        ans = max(ans,f[x]);
        d[x] = max(d[x],d[y]+w[i]);      //更新当前结点，下次循环计算
    }
}
```

特点：

优点：不仅可以求解正权值，也可以求解负权值。

缺点： 不方便记录直径的路径

U81904 【模板】树的直径

```cpp
#include<bits/stdc++.h>
using namespace std;
typedef long long LL;
const int N=500010;
int h[N],e[N*2],w[N*2],ne[N*2],idx;
void add(int x,int y,int z) {
    e[++idx]=y; w[idx]=z; ne[idx]=h[x]; h[x]=idx;
}
LL d[N],f[N],ans;
bool vis[N];
int n;

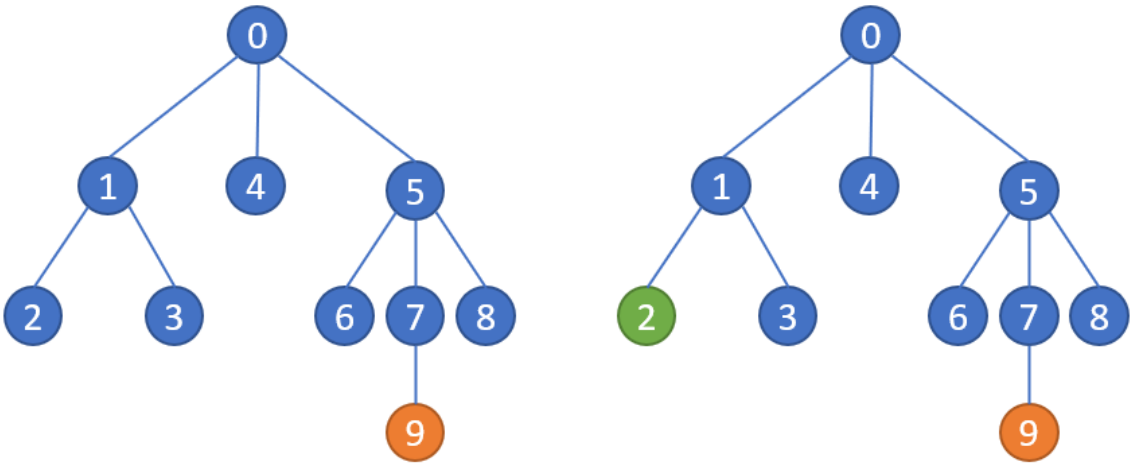
void dp(int x) {
    vis[x]=true;
    for(int i=h[x];i;i=ne[i]) {
        int y=e[i], z=w[i];
        if(vis[y])  continue;
        dp(y);
        f[x]=d[x]+d[y]+z;
        ans=max(ans,f[x]);
        d[x]=max(d[x], d[y]+z);
    }
}
int main() {
    scanf("%d",&n);
    for(int i=1;i<n;i++) {
        int x,y,z;  scanf("%d%d%d",&x,&y,&z);
        add(x,y,z); add(y,x,z);
    }
    dp(1);

    cout<<ans;

    return 0;
}
```

两次BFS或DFS求树的直径

通过两次遍历找出树的一条直径。



两次搜索，求树的直径

1. 从任意一个结点出发，通过DFS或BFS对树进行一次遍历，找出距离某个结点（例如根结点）最远的一个点p，比如结点（ 9 ）；
2. 从结点p出发，通过DFS或BFS再进行一次遍历，找出距离结点p最远的一个点q，比如结点（ 2 ）；

从p到q的路径，即为树的一条直径（$2-1-0-5-7-9$），长度为：6。这是因为p一定是直径的一段，否则总能找到一条更长的链，与直径的定义矛盾（可用反证法证明）

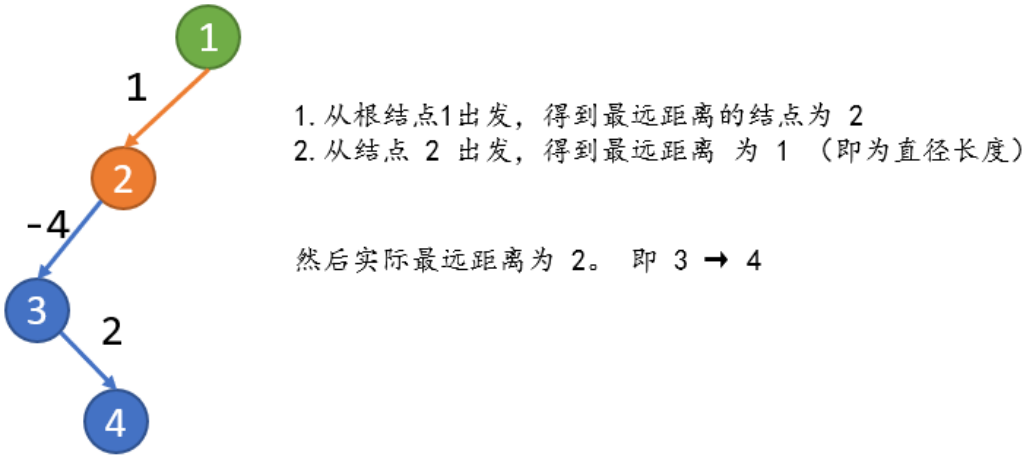
在第2步的遍历过程中，可用记录下来每个点第一次被访问时的前驱结点。最后从q递归回到p，即可得到直径的具体方案。

注意：为了找出距离某个点最远的点，这棵树应该看作无根树（即双向存储）。

特点：

优点：可以通过一个新的数组记录路径信息(例如父节点与子节点之间的关系)

缺点：**无法处理负权边**



树的直径可用作为很多树上问题的突破口。

树的直径相关性质：

1. 若有多条直径，则所有的直径之间皆有公共点
2. 直径的两个端点一定是叶子
3. 树中距离某一直径端点最远的点，至少有一个是该直径的另一个端点
4. 对树上任意一个点，与之距离最远的每一个点，至少有一个直径的端点

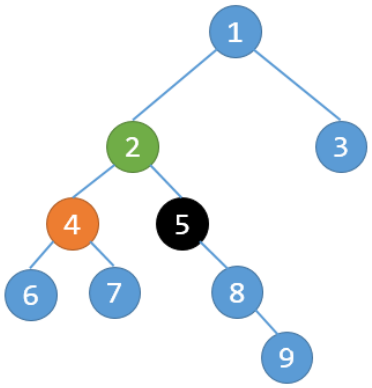
树的中心

定义：以树的中心为整棵树的根时，从该根到每个叶子节点的**最长路径最短**。

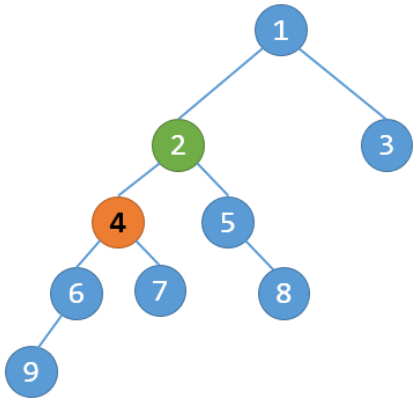
树结点的最远距离

题目描述： 从一个结点出发计算其最长路径。可以从这个结点出发，做一次BFS，每次扩展邻居结点，并记录到这个结点的最长距离，复杂度是$O(n)$，如果要求所有的n个结点的最长距离，需要对每个结点单独做一次BFS,总复杂度是$O(n^2)$。但是题目规模是$n \leq 100,000$，时间复杂度需要优化$O(nlogn)$。下面用树形DP的方式求解。

思路 ： 结点 u 的最长距离 $= max$ (向下走的最长距离， 向上走的最长距离)。结合下图我们来详细分析下过程：



情况1



情况2

1. 假设结点4 向下走的最长距离为 $d1[4]=1$ ， 次长距离为 $d2[4]=1$。 这两个数值我们可以通过树形DP的思想求解，时间复杂度是$O(n)$。
2. 假设结点4 向上走的最长距离为$up[2]$，求解$up[2]$的时候我们采用DFS,从根结点算下来即可。注意，现在是不是比较$max(d1[4],\ up[2]+dis(2,4))$ 呢？ 很明显不行。分两种情况：
    1. 如果结点 2 向下的最长路径经过结点 5，与当前子结点 4 不相符，那么就通过最长路径来更新最大值 $up[2]=max(d1[2],up[2])+dis(2,4)$
    2. 如果结点 2 向下的最长路径经过的结点 4，与当前子结点 4 相符，那么就通过次长路径来更新最大值$up[2]=max(d2[2],up[2])+dis(2,4)$

    还有一个问题，怎么记录最长路径经过的结点呢？这个也比较简单，创建一个数组$p[\ ]$，在计算最大值的时候顺便记录下结点即可。

代码参考：

```
#include<bits/stdc++.h>
using namespace std;
const int N=10010,M=N*2;
int h[N], e[M], w[M], ne[M], idx;

//d1表示向下的最长距离， d2表示向下的次长路径，up表示向上的最长路径，p表示最长路径的路径
```

```cpp
int d1[N],d2[N],up[N],p[N];
int f[N];   //每个结点的最远距离
int ans=1e9;    //计算最长距离的最小值，初始化无穷大
int n;

void add(int x,int y,int z) {       //链式前向星存图
    e[++idx]=y; w[idx]=z; ne[idx]=h[x]; h[x]=idx;
}
int dfs_down(int x,int fa) {
    // d1[x] = d2[x] = -1e9;    //初始化无穷小
    for(int i=h[x];i;i=ne[i]) {
        int y=e[i], z=w[i];
        if(y==fa)   continue ;  //不能往回走
        int dis = dfs_down(y,x) + z;

        // 判断最长距离和次长距离
        if(dis>d1[x]) {     //如果有更长的距离则更新，同时更新路径结点
            d2[x] = d1[x];  d1[x]=dis;  p[x]=y;
        }
        else if(dis>d2[x]){
            d2[x] =dis;
        }
    }
    // if(d1[x]==-1e9)  d1[x]=d2[x]=0;  //如果该点是叶子节点就返回0
    return d1[x];   //返回当前结点的最长距离
}
void dfs_up(int x,int fa) {
    for(int i=h[x];i;i=ne[i]) {
        int y=e[i], z=w[i];
        if(y==fa)   continue;   // 不能往回走
        //up[y] = up[x]+z;      //但是这里要分两种情况

        //分两种情况:
        if(p[x]==y) up[y]=max(up[x],d2[x])+z;
        else up[y] = max(up[x],d1[x])+z;
        dfs_up(y,x);        //注意这里dfs_up 与 dfs_down 的遍历方式不一样
    }
    return  ;
}
int main() {
    scanf("%d",&n);
    for(int i=1;i<n;i++) {
        int x,y,z;  scanf("%d%d%d",&x,&y,&z);
        add(x,y,z); add(y,x,z);     //无向边存图方式
    }
    dfs_down(1,0);      //求向下的最长路径和次长路径，这一步跟树形DP求直径一样

    dfs_up(1,0);        //求向上的最长路径

    for(int i=1;i<=n;i++)   f[i]=max(d1[i],up[i]);
    for(int i=1;i<=n;i++)   ans=min(ans,f[i]);
    cout<<ans;
    return 0;
}
```

当然，如果权值为负数的话，需要对代码细微的调整。

例题推荐

P5536 【XR-3】核心城市

P1099 [NOIP2007 提高组] 树网的核

P4408 [NOI2003] 逃学的小孩

P6722 「MCOI-01」Village 村庄

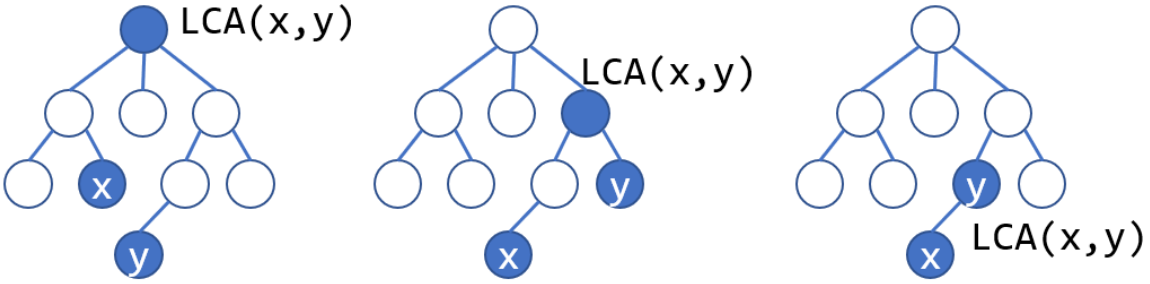
P7238 「DCOI」迷失森林

P3629 [APIO2010]巡逻

SP1437 Longest path in a tree （树的直径模板题）

最近公共祖先LCA

定义：给定一颗有根树，若节点z既是节点x的祖先，也是节点y的祖先，则称z是x,y的公共祖先。在x,y的所有公共祖先中，深度最大的一个称为x,y的最近公共祖先，记为$LCA(x,y)$。



LCA有以下性质：

1. 在所有公共祖先中，$LCA(x,y)$ 到 x,y的距离都最短。
2. x,y之间最短的路径，经过$LCA(x,y)$。
3. x,y本身也可以是它们自己的公共祖先。

LCA的求解思路：

简单直接的方法：分别从x 和y出发，一直往根结点走，第一次相遇的结点，就是$LCA(x,y)$。具体实现时，可以用标记法：首先从x出发一直向根结点走，沿路标记所有经过的祖先结点；把x的祖先标记完之后，然后再从y出发向根结点走，走到第一个被x标记的结点，就是$LCA(x,y)$。

标记法的复杂度较高，在有n个结点的树上求一次$LCA(x,y)$的计算量为$O(n)$。若有m次查询，总复杂度是$O(nm)$。效率太低。

经典的算法有倍增法、$Tarjan$算法（DFS+并查集），都能高效的求得LCA，适合做大量的查询。倍增法的复杂度是$O(nlogn+mlogn)$，相当好了。倍增是“在线算法”，单独处理每个询问。

$Tarjan$算法的复杂度是$O(m+n)$，是最优的算法，不可能更好了。$Tarjan$算法是“离线算法”，需要统一处理所有询问。

另外，树链剖分也是求LCA的常用方法。（感兴趣的同学可以往后学习）

树上倍增

前面提到的标记法可以换个方式实现，具体来说是以下两个步骤：

步骤（1）：先把x和y提到相同的深度。例如x比y深，就把x提到y的高度（即让x走到y的同一高度），如果发现y就是x的祖先，那么$LCA(x,y)=y$，停止查找，否则继续下一步。

步骤（2）：让x和y同步往上走，每走一步就判断是否相遇，相遇点就是$LCA(x,y)$，停止。

上面两个步骤，由于x和y都是慢腾腾一步一步往上走，复杂度都是$O(n)$的，如何改进？如果不是一步步走，而是跳着往上走，就能加快速度。如何跳？可以按2的倍数往上跳，跳$1,2,4,8,\ldots$,步，这就是倍增法。倍增法是常见的思路，应用很广，树上倍增求LCA是一个典型的应用。

倍增法用**“跳”**的方法加快了上面的两个步骤。注意已知条件是：每个结点知道它的子结点和父结点，并通过DFS 计算出了每个结点在树上的深度。下面仍然按照这两个步骤解释具体算法：

步骤（1）：把x和y提到相同的深度。具体任务是：给定两个结点x,y，设x比y深，让x **跳** 到与y相同的深度。注意x和y都是随机给定的，它们不是树上的特殊结点。

因为已知条件是只知道每个结点的父结点，所以如果没有其他辅助条件，x只能一步步往上走，没办法 **跳**。要实现 **跳** 的动作，必须提前计算出一些x的祖先结点，作为x 的 **跳板**。然而，应该提前计算出哪些祖先结点呢？通过这些预计算出的结点，真的能准确地跳到一个任意给定的y吗？最关键的是，这些预计算是高效的吗？这就是倍增法的精妙之处：预计算出每个结点的第$1,2,4,8,16,\ldots$个祖先，即以2倍增的祖先。

有了预计算出的这些祖先做跳板，能从x快速跳到任何一个给定的目标深度。注意，跳的时候先用大数。以从x跳到它的第27个祖先为例：

1. 从x跳 16步，到达x的第16个祖先$fa1$ ；
2. 从$fa1$ 跳 8 步，到达$fa1$的第8个祖先$fa2$ ；
3. 从$fa2$ 跳 2 步，到达$fa2$的第2个祖先$fa3$；
4. 从$fa3$ 跳 1 步，到达祖先$fa4$。

共跳了$18+8+2+1=27$。这个方法利用了二进制的特征：任何一个数都可以由2的倍数相加得到。而$27=（11011）_2$，其中的4个 **1** 的权值就是 $16,8,2,1$。把一个数转换为二进制数时，是从最高位往最低位转换的，这就是为什么要先用大数再用小数的原因。

显然，用倍增法从x跳到某个y的复杂度是$O(logn)$的。

剩下的问题是如何快速预计算每个结点这些倍增的祖先。定义$fa[x][i]$为x的第2^i个祖先，有以下非常巧妙的递推关系：$fa[x][i]=fa[fa[x][i-1]][i-1]$。

递推式的右边这样理解：

- $fa[x][i-1]$。从x起跳，先跳2^{i-1}步到了祖先$z=fa[x][i-1]$
- $fa[fa[x][i-1]][i-1]=fa[z][i-1]$。再从$z$ 跳2^{i-1}步到了祖先$fa[z][i-1]$。

一共跳了$2^{i-1}+2^{i-1}=2^i$ 步。公式右边实现了从x起跳，跳到了x的第2^i个祖先，这就是递推式左边的$fa[x][i]$。

特别地，$fa[x][0]$是x的第$2^0=1$个祖先，就是x的父结点。$fa[x][0]$是递推式的初始条件，从它递推出了所有的$fa[x][i]$。递推的计算量有多大？从任意一个结点x到根结点，最多只有$logn$个$fa[x][\,]$，所以只需要递推$O(logn)$次。计算n个结点的$fa[][]$，共计算$O(nlogn)$次。

步骤（2）： x和y同步往上跳，找到$LCA(x,y)$

经过步骤（1），x和y现在位于同一个深度，让它们同步往上跳，就能找到他们的公共祖先。x,y的公共祖先有很多，$LCA(x,y)$是距离x,y最近的那个，其他祖先都较远。以下的讨论都假设x和y深度相同。

能利用$fa[\,][\,]$来找$LCA(x,y)$吗？显然，$LCA(x,y)$ 并不一定正好位于$fa[x][\,]$和$fa[y][\,]$上，那么还能利用$fa[\,][\,]$数组吗？答案是确定的，其原理也用到了二进制的特征。

从一个结点跳到根结点，最多跳$logn$ 次。现在从x,y出发，从最大的$i=logn$ 开始，跳2^i步，跳到了祖先$fa[x][i]$、$fa[y][i]$，它们位于非常靠近根结点的位置$(2^i\approx 2^{logn}\approx n)$。有两种情况：

1. $fa[x][i]=fa[y][i]$，这是一个公共祖先，它的深度小于等于$LCA(x,y)$，这说明跳过头了，退回去换个小的$i-1$重新跳一次。

2. $fa[x][i] \neq fa[y][i]$，说明还没跳到公共祖先，那么更新$x = fa[x][i], y = fa[y][i]$，从新的起点$x, y$继续开始跳。由于新的$x, y$的深度比原来位置的深度减少超过一半，这样再跳的时候，就不用再跳2^i步，跳2^{i-1}步就够了。

以上两种情况，分别是比$LCA(x,y)$的浅和深的两种位置。用i循环判断以上两种情况，就是从深和浅两头逐渐逼近$LCA(x,y)$。每循环一次，$i-1$，当$i=0$时，x, y正好位于LCA的下一层，父结点$fa[x][0]$就是$LCA(x,y)$。

倍增法的计算包括预计算$fa[\,][\,]$和查询m次LCA,总复杂度是$O(nlogn + mlogn)$。

P3379 最近公共祖先（LCA）

题目描述：给定一棵有根多叉树，请求出指定两个点直接最近的公共祖先。

输入格式：第一行包含三个正整数N, M, S，分别表示树的结点个数、询问的个数和树根结点的序号。 接下来$N-1$行每行包含两个正整数x, y，表示x结点和y 结点之间有一条直接连接的边（数据保证可以构成树）。 接下来M行每行包含两个正整数a, b，表示询问a结点和b结点的最近公共祖先。

输出格式：输出M行，每行包含一个正整数，依次为每一个询问的结果。

数据规模：$N \leq 500000,\ M \leq 500000$。

代码参考：

```cpp
//树上倍增法
#include<cstdio>
#include<algorithm>
#include<cmath>
using namespace std;
const int N=500010;
int n,m,s;  //结点n，  询问m ，根结点s
int tot;
int d[N],f[N][22],h[N]; //d:分层，  f倍增跳跃
struct node {
    int u,v,next;   //邻接表
}e[2*N];
void add(int x,int y) { //构建邻接表
    e[++tot].u=x;
    e[tot].v = y;
    e[tot].next = h[x];
    h[x] = tot;
}
void dfs(int x) {   //寻找与x相连的y
    for(int i=h[x];i;i=e[i].next) {
        int y=e[i].v;
        if(d[y]==0) {   //如果y未被访问
            d[y] = d[x]+1;  //y是x的儿子，所以层数+1
            f[y][0] = x;    //y向上跳一格，就是x
            dfs(y);
        }
    }
}
int lca(int x,int y) {  //查找 x与 y的最近公共祖先
    if(d[x]<d[y])   swap(x,y);  //要求x比较深
    for(int i=20;i>=0;i--) {    //从2^k,2^k-1,.....,2^0开始递减跳
        if(d[f[x][i]]>=d[y]) {  //先让x跳到y的同层
            x=f[x][i];
        }
    }
```

```
    if(x==y)    return x;   //情况1： x的祖先是y
    //情况2：不重合
    for(int i=20;i>=0;i--) {
        if(f[x][i]!=f[y][i]) {      //如果祖先不相等,直到祖先相等
            x=f[x][i];  y=f[y][i];
        }
    }
    return f[x][0]; //x位于lca的下一层， 如果f[x][0]为最近公共祖先lca
}

int main() {
    scanf("%d %d %d",&n,&m,&s);
    int x,y;
    for(int i=1;i<n;i++) {
        scanf("%d %d",&x,&y);
        add(x,y);   add(y,x);   //存双向边
    }
    d[s]=1; f[s][0]=0;
    dfs(s); //建树
    for(int i=1;i<=20;i++) {
        for(int x=1;x<=n;x++) {
            f[x][i] = f[f[x][i-1]][i-1];
        }
    }
    for(int i=1;i<=m;i++) {
        scanf("%d %d",&x,&y);
        printf("%d\n",lca(x,y));
    }
    return 0;
}
```

Tarjan 算法

LCA的$Tarjan$ 算法 = “DFS + 并查集”， 是二者即简单又绝妙的组合。如果读者非常熟悉DFS和并查集，完全能自己推理出下面介绍的算法。

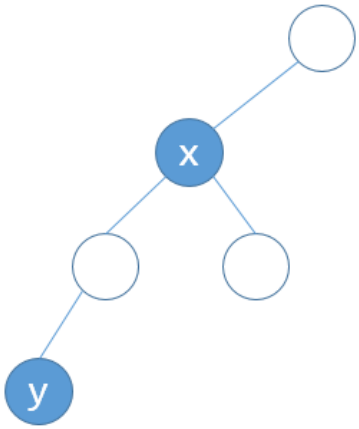
$Tarjan$ 算法是一种离线算法，它把所有的m个询问一次全部读入，统一计算，最后一起输出。$Tarjan$算法的效率极高，在n个结点的树上做m次LCA查询，总复杂度为$O(m+n)$，是可能达到的最优复杂度。

如何设计一种高效的离线算法？它和在线算法不一样，不一定要单独处理每个询问，而是有条件去全盘考虑所有的询问。如果把这些询问进行某种排序之后再计算，在整体上应该能得到较好的效率。如何排序？把一个询问(x,y)看成一对结点，那么就按x排序。在树这种情况下，用DFS遍历树时，按x出现的先后为序，每处理一个x结点，就查找与x有关的结点对(x,y)，计算$LCA(x,y)$。

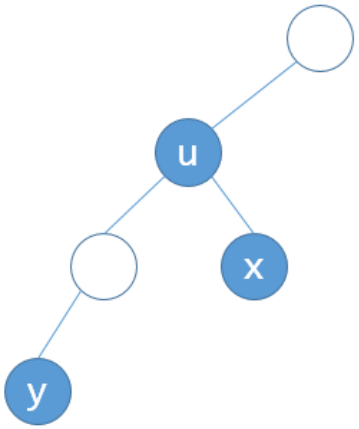
有多种DFS遍历方法，例如先序、中序、后序等，哪一种适合用来计算LCA？再次回顾标记法，它是从底层的x、y结点出发，逐步向高层的根结点走，直到第一次相遇，就是$LCA(x,y)$。DFS后序遍历应该很适合这种情况，后序DFS先返回最底层的叶子结点，而且是从底层结点逐层回溯到根结点，符合标记法的计算顺序。

现在以x为主，y为辅计算$LCA(x,y)$。 设现在遍历到了一个结点x，下面考虑结点对(x,y)的y。x和y只有两种关系：

1. y在x的子树上；
2. y不在x的子树上。



y在x的子树上



y不在x的子树上

（1）y在x的子树上。即y的祖先是x，有$LCA(x,y)=x$。具体编程时这样做：以x为DFS的入口，因为y是在x的子树上，所以DFS后序遍历回溯先返回y，标记y为已经访问过，记$vis[y]=true$；后面回溯到x时， 查询结点对(x,y)，若$vis[y]=true$，那么显然有$LCA(x,y)=x$。

（2）y不在x的子树上。设它们的公共祖先是u，以u为DFS的入口。DFS先访问到y，标记$vis[y]=true$，并在从y回溯到u的过程中，记录y的祖先结点是u，记为$fa[y]=u$。访问到x时，查询结点对(x,y)，若 $vis[y]=true$，那么有$LCA(x,y)=LCA(x,u)=u$。读者可能注意到，若DFS先访问到x，而不是y，如何处理？忽略即可，因为x和y是成对的，后面访问到y时，再以y为主，x为辅即可。 这两种情况可以合并。在第(1)种情况中，从y回溯到x时，记录y的祖先是x，即$fa[y]=x$，这是情况(2) 的特例。

上面的讨论，是以某个x为根，或者以某个u为根进行子树的遍历，计算出$LCA(x,y)$。能否扩展到整棵树，用一个DFS解决所有的LCA查询？

这就是$Tarjan$算法的基本思路：以树的根结点为DFS入口，遍历整棵树，每遍历到一个结点，就把它看成一个x，检查x的所有结点对(x,y)的y，若$vis[y]=true$且$fa[y]=u$，那么 $LCA(x,y)=u$。

最后还有一个关键问题没有解决：如何计算$fa[y]=u$？即如何在回溯过程中，把以结点u为根的子树上的所有子结点的祖先都设置为u？如果读者非常熟悉并查集，就能发现，一棵以u为根的子树，刚好是以u为集合的一个并查集。那么就容易编码了：从子树的一个结点y回溯时，把父结点$fa[y]$看成y的集。逐级回溯到根u的过程中，每个结点的集都记录为它的父结点。当查询y的集时，通过查找函数$find_set()$，最终查到y的集是u。

$Tarjan$算法的复杂度很好。每个结点只访问1次，每个询问也只处理一次，总复杂为$O(m+n)$，是可能达到的最优复杂度，不可能更好了。

```
#include<bits/stdc++.h>
using namespace std;
typedef long long LL;
const int N=500010;
int fa[N],ans[N];
bool vis[N];

int h[N],e[N*2],ne[N*2],idx;
int qh[N],qe[N*2],qn[N*2],qi[N*2],tot;  //询问

void add(int x,int y){
    e[++idx]=y; ne[idx]=h[x];   h[x]=idx;
}
void add_query(int x,int y,int num) {
    qe[++tot] = y;
```

```
    qi[tot] = num;       //存储询问的编号
    qn[tot] = qh[x];
    qh[x]=tot;
}
int find_set(int x) {    //没有路径压缩，保持树的原型
    return fa[x]==x ? x: find_set(fa[x]);
}
void tarjan(int x) {
    vis[x]=true;
    for(int i=h[x];i;i=ne[i]) {
        int y=e[i];
        if(!vis[y]) {
            tarjan(y);
            fa[y] =x;
        }
    }
    for(int i=qh[x];i;i=qn[i]) {     //查询与x有关的结点对
        int y=qe[i];
        //如果y在x的子树上，则vis[y]=true,lca(x,y)=find_set(y)
        if(vis[y]) {
            ans[qi[i]] = find_set(y);
        }
    }
}
int main() {
    int n,m,s;
    cin>>n>>m>>s;
    for(int i=1;i<=n;i++)    fa[i]=i;     //初始化集合
    for(int i=1;i<n;i++) {
        int x,y;     scanf("%d%d",&x,&y);
        add(x,y);    add(y,x);        //无向边
    }
    for(int i=1;i<=m;i++) {
        int x,y;     scanf("%d%d",&x,&y);
        add_query(x,y,i);    add_query(y,x,i);    //存储询问情况，存储方式与链式前向星一样
    }
    tarjan(s);
    for(int i=1;i<=m;i++)    cout<<ans[i]<<endl;
    return 0;
}
```

应用

LCA的最基本应用是求树上两个结点的最短距离，它等于两点深度之和减去两倍的LCA深度：
$dist[x,y] = deep[x] + deep[y] - 2 * deep[LCA(x,y)]$

LCA的另外一个经典应用就是树上差分。

树上差分

P3128 [USACO15DEC]Max Flow P

题目描述：有n个结点，用$n-1$条边连接，所有结点都连通了。给出m条路径，第i条路径从结点si到ti。每给出一条路径，路径上所有结点的权值加1。输出最大权值点的权值。

输入格式：第一行是n和m。后面$n-1$行，每行包括2个整数x,y，表示一条边。后面m行，每行2个整数s和t，表 示一条路径的起点和终点。

输出格式：输出一个整数，表示最大权值。

数据规模：$2 \leq n \leq 50,000$，$1 \leq m \leq 100,000$

算法分析：

树上两点u、v的路径，显然是最短路径。把$u \to v$路径分为两部分：$u \to LCA(u,v)$和$LCA(u,v) \to v$。 先考虑简单的思路。首先对每个路径求LCA，分别以u和v为起点到LCA，把路径上每个结点的权值加 1；然后对所有m个路径进行类似操作。把路径上每个结点加1操作的复杂度是$O(n)$，再乘上m次求LCA的时间，总时间为$O(nm)$，会超时。 本题的关键是如何记录路径上每个结点的修改。显然，如果真的对每个结点都记录修改，肯定会超时。此时可以利用差分，差分的重要用途是**“把区间问题转换为端点问题”**，正适合这种情况。

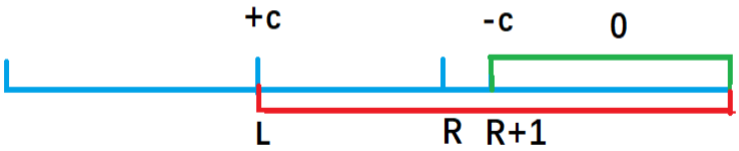
给定数组a[]，定义差分数组：$d[k] = a[k] - a[k-1]$，即数组相邻元素的差。

从定义推出：$a[k] = d[1] + d[2] + d[3] + \ldots + d[k] = \sum_{i=1}^{k} d(i)$

这个公式描述了a和d 的关系：“**差分是前缀和的逆运算**”，它把求$a[k]$转化为求d的前缀和。

对于区间$[L,R]$的修改问题，例如把区间内每个元素加上c。对区间的两个端点做以下操作：

1. 把$d[L] + c$
2. 把$d[R+1] - c$。



然后求前缀和$sum[x] = d[1] + d[2] + \ldots + d[x]$， 有：

1. $1 \leq x < L$，前缀和$sum[x]$不变；
2. $L \leq x \leq R$，前缀和$sum[x]$增加c；
3. $R < x \leq N$，前缀和$sum[x]$不变，因为被$d[R+1]$中减去的c抵消了；

$sum[x]$ 等于$a[x]$，这样就可以利用差分数组计算出的区间修改后的$a[x]$。

从以上讨论得到一个关键的方法：利用差分，能够把区间修改问题转换为只用端点做记录。不用差分数组时，区间内每个元素都需要修改，复杂度$O(n)$；用差分转换为只记录两个端点后，复杂度减少到$O(1)$。 这就是差分的重要作用。

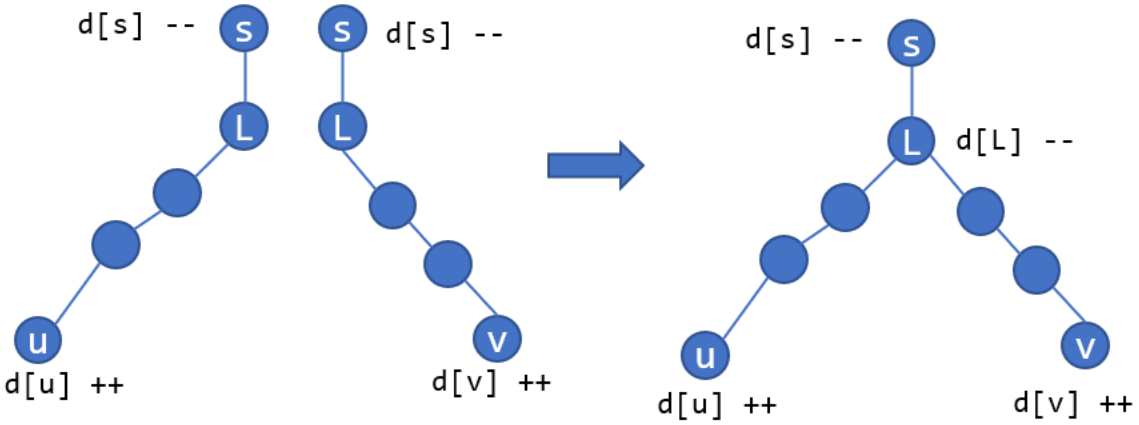
把上述的差分概念应用在树上，只需要把树上路径转换为区间即可。把一条路径$u \to v$分为两部分：$u \to LCA(u,v)$和$LCA(u,v) \to v$，这样每个路径都可以当成一个区间来处理。

记$LCA(u,v) = L$，并记L的父结点为$s = fa[L]$，本题是把路径上每个结点权值加1：

（1）路径$u \to L$这个区间上，$d[u]++$，$d[s]--$。

（2）路径$L \to v$这个区间上，$d[v]++$，$d[s]--$。

经过以上操作，能通过$d[]$计算出$u \to v$上每个结点的权值。不过，由于两个路径在L和S这里重合了，上面2个步骤把$d[L]$加了2次，把$d[s]$减了2次，需要调整为：$d[LCA(u,v)]--$和$d[s]--$。



在本题中，对每个路径都用倍增法求一次LCA，并做一次差分操作。当所有路径都计算完之后，再做一次DFS，求出每个结点的$sum[]$，即求得每个结点的权值。其中的最大值为答案。

复杂度讨论：m次LCA复杂度$O(nlogn+mlogn)$；最后做一次DFS，复杂度$O(n)$；总复杂度约$O(mlogn)$

代码参考：

```cpp
#include<bits/stdc++.h>
using namespace std;
const int N=50010;
int h[N],e[N*2],ne[N*2],idx;
int d[N],deep[N],fa[N][21],ans,cnt;
int n,m,s;
bool vis[N];
void add(int x,int y){
    e[++idx]=y; ne[idx]=h[x]; h[x]=idx;
}
void dfs1(int x) {  //搜索整棵树，计算出深度和初始化fa
    for(int i=h[x];i;i=ne[i]) {
        int y=e[i];
        if(deep[y]==0){
            deep[y]=deep[x]+1;
            fa[y][0]=x; //跳一步就是x
            dfs1(y);    //搜索下一个结点
        }
    }
    return ;
}
int LCA(int x,int y) {
    if(deep[x]<deep[y]) swap(x,y);  //保证x结点在最深的位置
    for(int i=20;i>=0;i--){
        if(deep[fa[x][i]]>=deep[y]) {
            x = fa[x][i];
        }
    }
    if(x==y)    return x;   //y为x的祖先元素
    for(int i=20;i>=0;i--) {    //现在x,y都处于同一层位置
        if(fa[x][i] != fa[y][i]) {
            x=fa[x][i]; y=fa[y][i];
        }
    }
    return fa[x][0];
}
```

```cpp
void dfs2(int x) {
    vis[x]=true;
    for(int i=h[x];i;i=ne[i]) {
        int y=e[i];
        if(vis[y]==0) {
            dfs2(y);
            d[x] += d[y];
        }
    }
    ans = max(ans,d[x]);     //计算最大权值的点
    return ;
}
int main() {
    scanf("%d%d",&n,&m);
    for(int i=1;i<n;i++){
        int x,y;     scanf("%d%d",&x,&y);
        add(x,y);    add(y,x);
    }
    int s=1;
    fa[s][0]=0, deep[s]=1;
    dfs1(s);     //搜索整棵树，求出每个结点的深度和初始化的fa

    for(int i=1;i<=20;i++) {
        for(int x=1;x<=n;x++) {
            fa[x][i]=fa[fa[x][i-1]][i-1];
        }
    }

    for(int i=1;i<=m;i++) {
        int x,y;     scanf("%d%d",&x,&y);
        int lca=LCA(x,y);
        d[x]++; d[y]++; d[lca]--; d[fa[lca][0]]--;
    }
    dfs2(1);     //用差分数组一次求每个结点的权值
    cout<<ans;
    return 0;
}
```

例题推荐：

- P5836 [USACO19DEC]Milk Visits S
- P3884 [JLOI2009]二叉树问题
- P3938 斐波那契
- P3258 [JLOI2014]松鼠的新家
- P6869 [COCI2019-2020#5] Putovanje
- P5021 [NOIP2018 提高组] 赛道修建
- P1967 [NOIP2013 提高组] 货车运输
- P4427 [BJOI2018]求和
- P1600 [NOIP2016 提高组] 天天爱跑步
- P2680 [NOIP2015 提高组] 运输计划

【选读】树上分治——点分治

假如现在我们得到了一棵n个节点的树，每条边都有长度。现在我们要求这棵树中两个点之间距离小于k的点对个数。

朴素的做法是：先预处理好距离，再$O(n^2)$枚举点对。

而点分治可以把时间复杂度优化到$O(nlog^2n)$